

Parity bit check Vs CRC Vs Checksum. [combined 3 bank-2024]

Error Detection in Computer Networks

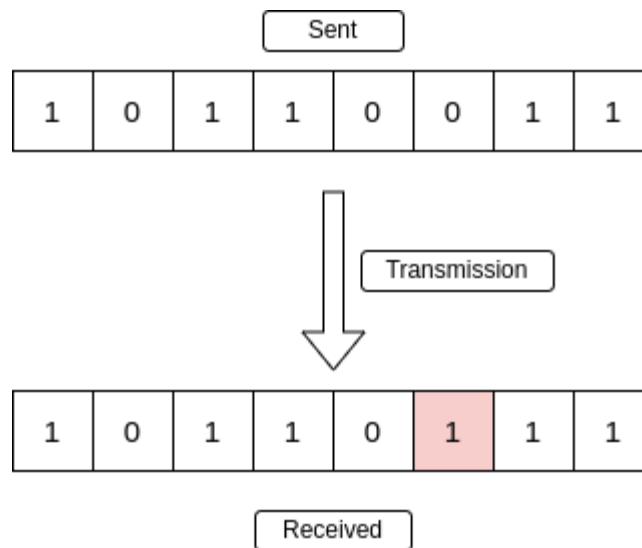
Error is a condition when the receiver's information does not match the sender's. Digital signals suffer from noise during transmission that can introduce errors in the binary bits traveling from sender to receiver. That means a 0 bit may change to 1 or a 1 bit may change to 0.

Data (Implemented either at the Data link layer or Transport Layer of the OSI Model) may get scrambled by noise or get corrupted whenever a message is transmitted. To prevent such errors, error-detection codes are added as extra data to digital messages. This helps in detecting any errors that may have occurred during message transmission.

Types of Errors

Single-Bit Error

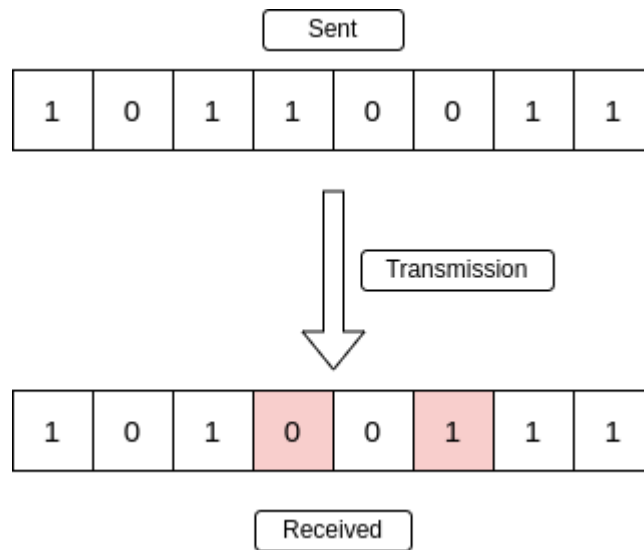
A single-bit error refers to a type of data transmission error that occurs when one bit (i.e., a single binary digit) of a transmitted data unit is altered during transmission, resulting in an incorrect or corrupted data unit.



Single-Bit Error

Multiple-Bit Error

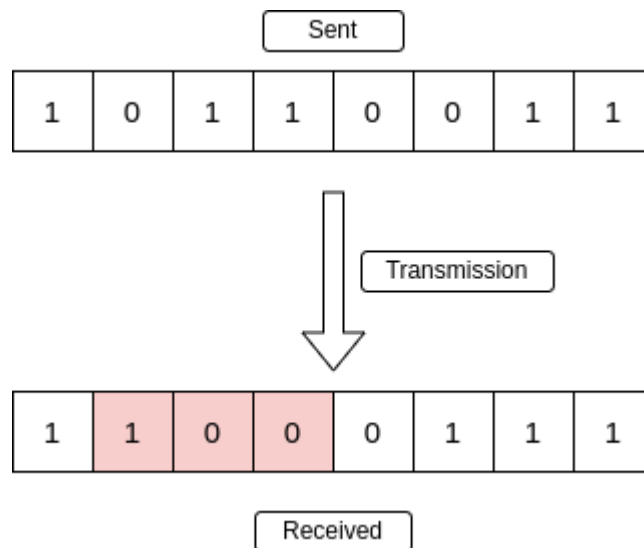
A multiple-bit error is an error type that arises when more than one bit in a data transmission is affected. Although multiple-bit errors are relatively rare when compared to single-bit errors, they can still occur, particularly in high-noise or high-interference digital environments.



Multiple-Bit Error

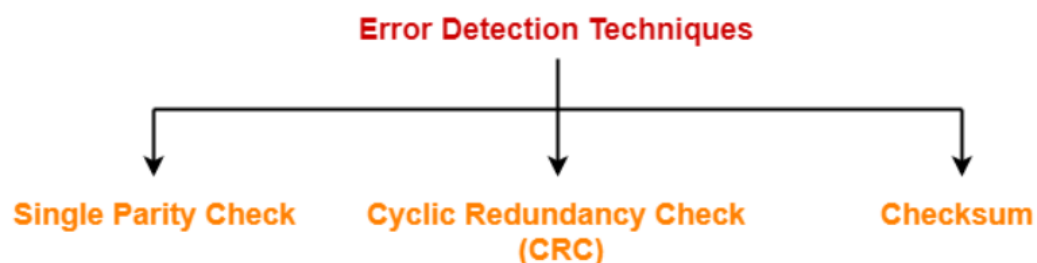
Burst Error

When several consecutive bits are flipped mistakenly in digital transmission, it creates a burst error. This error causes a sequence of consecutive incorrect values.



Burst Error

Error Detection Methods



Single Parity Check-

In this technique,

- One extra bit called as **parity bit** is sent along with the original data bits.
- Parity bit helps to check if any error occurred in the data during the transmission.

Steps Involved-

Error detection using single parity check involves the following steps-

Step-01:

At sender side,

- Total number of 1's in the data unit to be transmitted is counted.
- The total number of 1's in the data unit is made even in case of even parity.
- The total number of 1's in the data unit is made odd in case of odd parity.
- This is done by adding an extra bit called as **parity bit**.

Step-02:

- The newly formed code word (Original data + parity bit) is transmitted to the receiver.

Step-03:

At receiver side,

- Receiver receives the transmitted code word.
- The total number of 1's in the received code word is counted.

Then, following cases are possible-

- If total number of 1's is even and even parity is used, then receiver assumes that no error occurred.
- If total number of 1's is even and odd parity is used, then receiver assumes that error occurred.
- If total number of 1's is odd and odd parity is used, then receiver assumes that no error occurred.
- If total number of 1's is odd and even parity is used, then receiver assumes that error occurred.

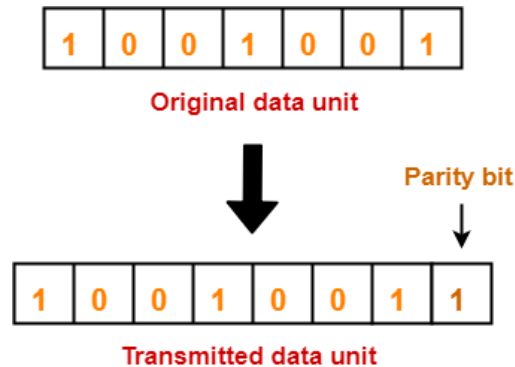
Parity Check Example-

Consider the data unit to be transmitted is 1001001 and even parity is used.

Then,

At Sender Side-

- Total number of 1's in the data unit is counted.
- Total number of 1's in the data unit = 3.
- Clearly, even parity is used and total number of 1's is odd.
- So, parity bit = 1 is added to the data unit to make total number of 1's even.
- Then, the code word 10010011 is transmitted to the receiver.



At Receiver Side-

- After receiving the code word, total number of 1's in the code word is counted.
- Consider receiver receives the correct code word = 10010011.
- Even parity is used and total number of 1's is even.
- So, receiver assumes that no error occurred in the data during the transmission.

Advantage-

- This technique is guaranteed to detect an odd number of bit errors (one, three, five and so on).
- If odd number of bits flip during transmission, then receiver can detect by counting the number of 1's.

Also Read- [Checksum](#)

Limitation-

- This technique can not detect an even number of bit errors (two, four, six and so on).
- If even number of bits flip during transmission, then receiver can not catch the error.

EXAMPLE

- Consider the data unit to be transmitted is 10010001 and even parity is used.

- Then, code word transmitted to the receiver = 100100011
- Consider during transmission, code word modifies as 101100111. (2 bits flip)
- On receiving the modified code word, receiver finds the number of 1's is even and even parity is used.
- So, receiver assumes that no error occurred in the data during transmission though the data is corrupted.

Advantages of Simple Parity Check

- Simple parity check can detect all single bit error.
- Simple parity check can detect an odd number of errors.
- **Implementation:** Simple Parity Check is easy to implement in both hardware and software.
- **Minimal Extra Data:** Only one additional bit (the parity bit) is added per data unit (e.g., per byte).
- **Fast Error Detection:** The process of calculating and checking the parity bit is quick, which allows for rapid error detection without significant delay in data processing or communication.
- **Single-Bit Error Detection:** It can effectively detect single-bit errors within a data unit, providing a basic level of error detection for relatively low-error environments.

Disadvantages of Simple Parity Check

- **Single Parity check is not able to detect even no. of bit error.**
- **For example,** the Data to be transmitted is **101010**. Codeword transmitted to the receiver is 1010101 (we have used even parity).
- Let's assume that during transmission, two of the bits of code word flipped to 1111101. On receiving the code word, the receiver finds the no. of ones to be even and hence **no error**, *which is a wrong assumption.*

Checksum

Explain the logic of a “Checksum”. How is it used to verify data integrity during file transfer? [AP (Bank)- 2026]

Checksum:

Checksum is an error detection method.

Error detection using checksum method involves the following steps-

Step-01: At sender side,

- If m bit checksum is used, the data unit to be transmitted is divided into segments of m bits.
- All the m bit segments are added.
- The result of the sum is then complemented using 1's complement arithmetic.
- The value so obtained is called as **checksum**.

Step-02:

- The data along with the checksum value is transmitted to the receiver.

Step-03: At receiver side,

- If m bit checksum is being used, the received data unit is divided into segments of m bits.
- All the m bit segments are added along with the checksum value.
- The value so obtained is complemented and the result is checked.

Then, following two cases are possible-

Case-01: Result = 0 If the result is zero,

- Receiver assumes that no error occurred in the data during the transmission.
- Receiver accepts the data.

Case-02: Result $\neq$ 0 If the result is non-zero,

- Receiver assumes that error occurred in the data during the transmission.
- Receiver discards the data and asks the sender for retransmission.

Checksum Example-

Consider the data unit to be transmitted is-

10011001111000100010010010000100

Consider 8 bit checksum is used.

Step-01: At sender side,

The given data unit is divided into segments of 8 bits as-

10011001	11100010	00100100	10000100
----------	----------	----------	----------

Now, all the segments are added and the result is obtained as-

- $10011001 + 11100010 + 00100100 + 10000100 = 1000100011$
- Since the result consists of 10 bits, so extra 2 bits are wrapped around.
- $00100011 + 10 = 00100101$ (8 bits)
- Now, 1's complement is taken which is 11011010.
- Thus, checksum value = 11011010

Step-02:

- The data along with the checksum value is transmitted to the receiver.

Step-03:

At receiver side,

- The received data unit is divided into segments of 8 bits.
- All the segments along with the checksum value are added.
- Sum of all segments + Checksum value = $00100101 + 11011010 = 11111111$
- Complemented value = 00000000
- Since the result is 0, receiver assumes no error occurred in the data and therefore accepts it.

Also Read- [Parity Check](#)

Important Notes-

Note-01:

- Consider while adding the m bit segments, the result obtained consists of more than m bits.
- Then, wrap around the extra bits and add to the result so that checksum value consists of m bits.

Note-02:

- While calculating the checksum, if checksum value is needed, then assume it to be zero.

- After calculating the checksum value, substitute the checksum value in the checksum field.
- This will be required during checksum calculation of [IP Header](#), [TCP Header](#) and [UDP Header](#).

Note-03:

- The checksum is used in the internet by several protocols although not at the data link layer.

Also Read- [Cyclic Redundancy Check \(CRC\)](#)

PRACTICE PROBLEM BASED ON CHECKSUM ERROR DETECTION METHOD-

Problem-

Checksum value of 1001001110010011 and 1001100001001101 of 16 bit segment is-

1. 1010101000011111
2. 1011111000100101
3. 1101010000011110
4. 1101010000111111

Solution-

We apply the above discussed algorithm to calculate the checksum.

- $1001001110010011 + 1001100001001101 = 10010101111100000$
- Since, the result consists of 17 bits, so 1 bit is wrapped around and added to the result.
- $0010101111100000 + 1 = 0010101111100001$
- Now, result consists of 16 bits.
- Now, 1's complement is taken which is 1101010000011110
- Thus, checksum value = 1101010000011110

Thus, Option (C) is correct.

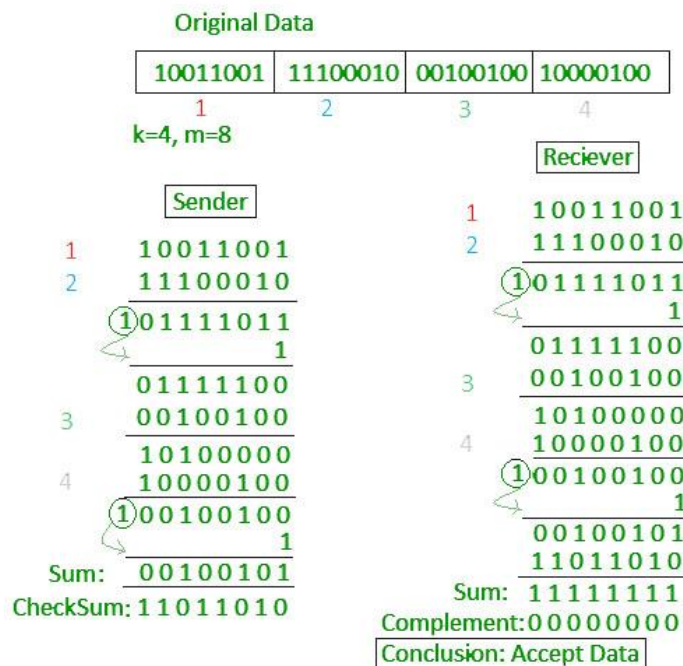
To gain better understanding about Checksum Method,

Example 2 - If the data transmitted along with checksum is 10101001 00111001 00011101. But the data received at destination is 00101001 10111001 00011101.

Receiver Site:

00101001 1st bit of subunit 1 is damaged
10111001 1st bit of subunit 2 is damaged
00011101 checksum
11111111 sum
00000000 Ok 1's complement

Although data is corrupted, the error is undetected. Because the changes in data 1 and data 2 offset each other, resulting in the same overall sum. **This is a limitation of simple checksum – it cannot detect all types of errors, especially when multiple errors cancel each other out.**



Advantages of Checksum [Read more about Checksum](#)

- Simple: Checksum is very easy to implement and consumes very less computational power.
- Fast Execution: It contains simple addition so execution of the checksum operation takes place rapidly.
- Usage in Simple Systems: Due to its simplicity, it is applied for error detection in basic protocols such as IPv4.

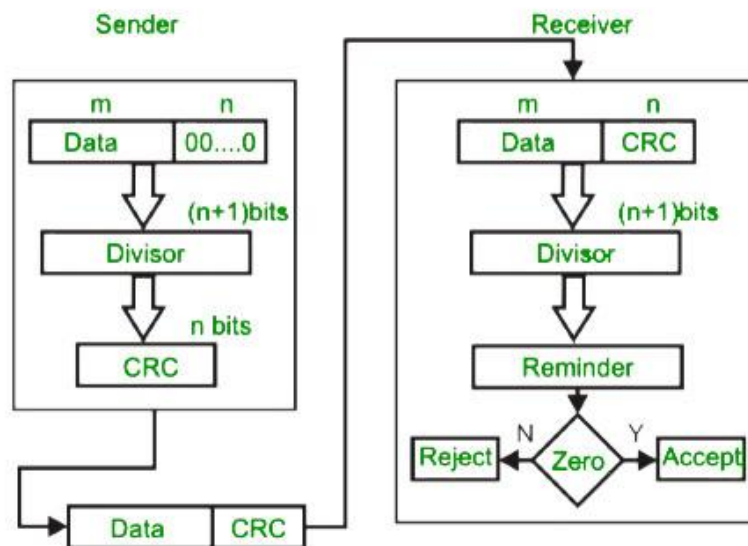
Disadvantages of Checksum

- Low Precision: Checksum fails in the case of some error types where errors cancel each other out. For example, flipping two bits can result in the same sum.

- **Limited Error Detection:** This cannot detect intricate or multi-bit errors, making it less reliable for large data blocks or for applications that require high integrity.

Cyclic Redundancy Check (CRC)

- Unlike the checksum scheme, which is based on addition, CRC is based on [binary division](#).
- In CRC, a sequence of redundant bits, called cyclic redundancy check bits, are appended to the end of the data unit so that the resulting data unit becomes exactly divisible by a second, predetermined binary number.
- At the destination, the incoming data unit is divided by the same number. If at this step there is no remainder, the data unit is assumed to be correct and is therefore accepted.
- A remainder indicates that the data unit has been damaged in transit and therefore must be rejected.



CRC Working

We have given dataword of length n and divisor of length k .

Step 1: Append $(k-1)$ zero's to the original message

Step 2: Perform modulo 2 division

Step 3: Remainder of division = CRC

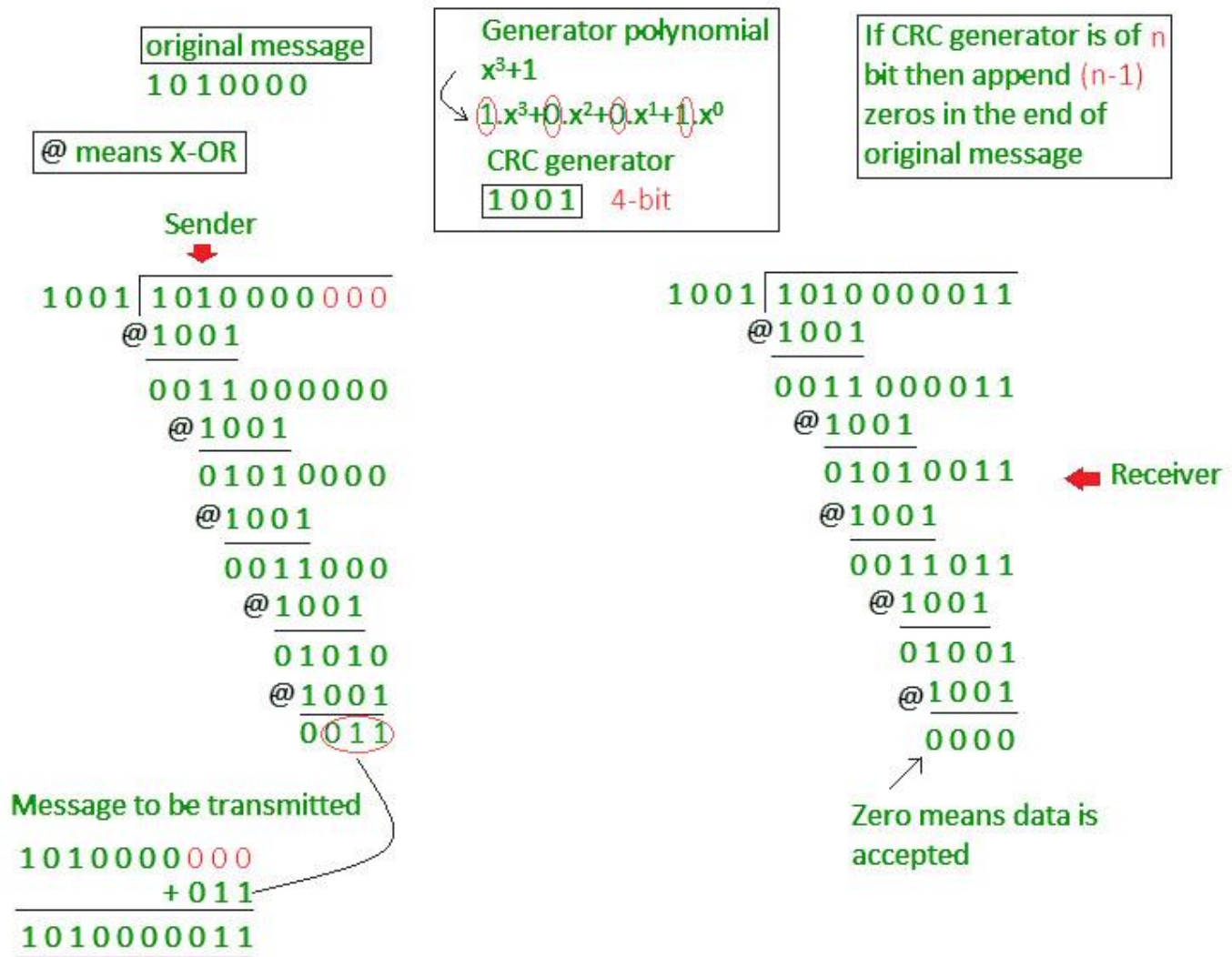
Step 4: Code word = Data with append $k-1$ zero's + CRC

Note:

- CRC must be $k-1$ bits

- Length of Code word = $n+k-1$ bits

Example: Let's data to be send is 1010000 and divisor in the form of polynomial is x^3+1 . CRC method discussed below.



Example: Previous year GATE questions based on error detection: [GATE CS 2009 Question 48](#) [GATE CS 2007 Question 68](#).

RACTICE PROBLEMS BASED ON CYCLIC REDUNDANCY CHECK (CRC)-

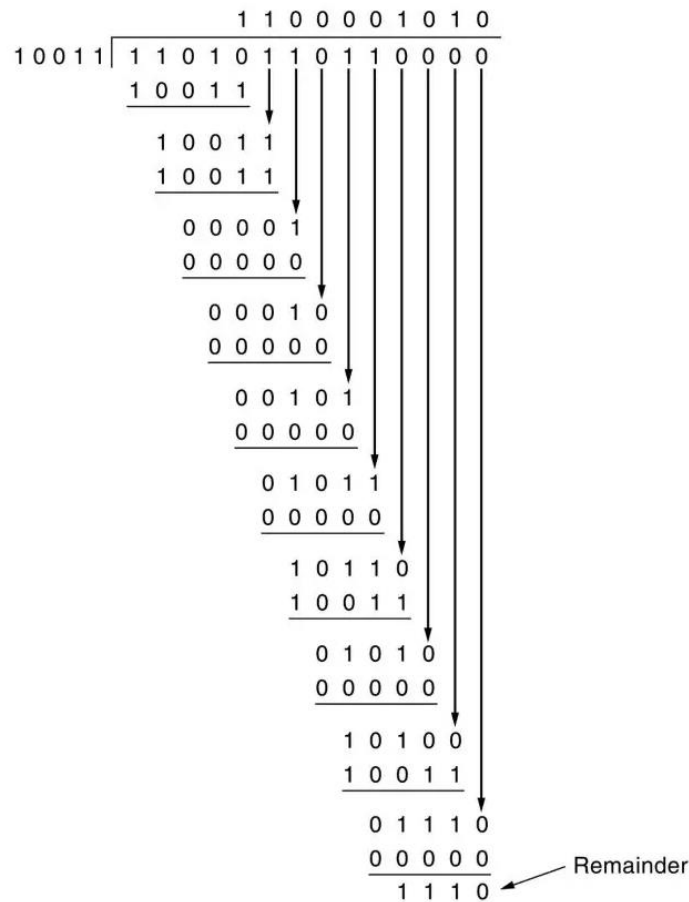
Problem-01: A bit stream 1101011011 is transmitted using the standard CRC method. The generator polynomial is x^4+x+1 . What is the actual bit string transmitted?

Solution-

- The generator polynomial $G(x) = x^4 + x + 1$ is encoded as 10011.
- Clearly, the generator polynomial consists of 5 bits.
- So, a string of 4 zeroes is appended to the bit stream to be transmitted.

- The resulting bit stream is 1101011011**0000**.

Now, the binary division is performed as-



From here, CRC = 1110.

Now,

- The code word to be transmitted is obtained by replacing the last 4 zeroes of 1101011011**0000** with the CRC.
- Thus, the code word transmitted to the receiver = 1101011011**1110**.

Problem-02:

A bit stream 10011101 is transmitted using the standard CRC method. The generator polynomial is x^3+1 .

1. What is the actual bit string transmitted?
2. Suppose the third bit from the left is inverted during transmission. How will receiver detect this error?

Solution-

Part-01:

- The generator polynomial $G(x) = x^3 + 1$ is encoded as 1001.
- Clearly, the generator polynomial consists of 4 bits.
- So, a string of 3 zeroes is appended to the bit stream to be transmitted.
- The resulting bit stream is 10011101**000**.

Now, the binary division is performed as-

$$\begin{array}{r} 10001100 \\ 1001 \overline{) 10011101000} \\ \underline{1001} \\ 00001 \\ \underline{0000} \\ 00011 \\ \underline{0000} \\ 00110 \\ \underline{0000} \\ 01101 \\ \underline{1001} \\ 01000 \\ \underline{1001} \\ 00010 \\ \underline{0000} \\ 00100 \\ \underline{0000} \\ 0100 \end{array} \leftarrow \text{CRC}$$

From here, CRC = 100.

Now,

- The code word to be transmitted is obtained by replacing the last 3 zeroes of 10011101**000** with the CRC.
- Thus, the code word transmitted to the receiver = 10011101**100**.

Part-02:

According to the question,

- Third bit from the left gets inverted during transmission.
- So, the bit stream received by the receiver = 10111101100.

Now,

- Receiver receives the bit stream = 10111101100.
- Receiver performs the binary division with the same generator polynomial as-

$$\begin{array}{r}
 10101000 \\
 1001 \overline{) 10111101100} \\
 \underline{1001} \\
 00101 \\
 \underline{0000} \\
 01011 \\
 \underline{1001} \\
 00100 \\
 \underline{0000} \\
 01001 \\
 \underline{1001} \\
 00001 \\
 \underline{0000} \\
 00010 \\
 \underline{0000} \\
 00100 \\
 \underline{0000} \\
 0100 \leftarrow \text{Remainder}
 \end{array}$$

From here,

- The remainder obtained on division is a non-zero value.
- This indicates to the receiver that an error occurred in the data during the transmission.
- Therefore, receiver rejects the data and asks the sender for retransmission.

Important Notes-

If the CRC generator is chosen according to the above rules, then-

- CRC can detect all single-bit errors
- CRC can detect all double-bit errors provided the divisor contains at least three logic 1's.
- CRC can detect any odd number of errors provided the divisor is a factor of $x+1$.

- CRC can detect all burst error of length less than the degree of the polynomial.
- CRC can detect most of the larger burst errors with a high probability.

Advantages of CRC

- **Higher Sensitivity:** CRC is much more sensitive and can identify more complex error patterns, including burst errors. It is much more reliable for a large amount of data.
- **Used in Crucial Systems:** CRC is used more frequently in data communication protocols (such as Ethernet, USB) and storage devices because of its ability.
- **Multiple cases of errors:** It can identify multiple patterns of error types, such as multiple bit errors and patterns that checksums cannot detect.

Disadvantages of CRC

- **Complexity:** CRC requires more computational resources to implement and execute than a simple checksum.
- **Impact on Performance:** The division used in the CRC algorithm can cause an overhead in data processing in some real-time systems.

Example2:

What is CRC?

CRC or [Cyclic Redundancy Check](#) is a more sophisticated than a checksum. It works on the principle that divide the data, treated as a polynomial, by a specified polynomial leaving a remainder called the CRC code. The CRC code is attached to the data. On the receiving end, the same polynomial division is done; if the remainder matches the CRC code then this data is valid. It contains **Polynomial Generator** on both sender and receiver side. The polynomial generator is of the type x^3+x^2+x+1 .

Difference between Checksum and CRC

Feature	Checksum	Cyclic Redundancy Check (CRC)
Definition	A simple error-detection method that sums up data chunks and verifies the integrity based on the sum.	A more advanced error-detection method using polynomial division to calculate a remainder for error checking
Error Detection Method	Simple addition of data values	Polynomial division of data treated as a binary number or polynomial

Accuracy	Low accuracy; may miss some errors, especially if they cancel each other out	High accuracy; detects most error patterns, including burst and multiple-bit errors
Complexity	Simple to compute, requires minimal resources	More complex; involves mathematical operations (polynomial division)
Error Detection Capability	Detects simple, single-bit errors	Detects complex errors, including burst errors and multiple-bit errors
Computation Time	Faster to compute due to simplicity	Slower to compute due to the more complex division operation
Overhead	Minimal (small data overhead)	Slightly higher overhead due to additional CRC code
Use Cases	Used in simple protocols (e.g., IPv4, UDP)	Used in critical systems (e.g., Ethernet, USB, hard drives, wireless communication)
Reliability	Less reliable, especially for larger datasets or complex errors	Highly reliable for detecting a wide range of errors
Performance Impact	Low, ideal for simple systems with limited resources	Higher, especially in real-time systems with large data streams

Which method is better for detecting errors, Checksum or CRC?

CRC is more effective for detecting errors as it can catch more complex and multiple-bit errors compared to Checksum.

Why is Checksum less reliable than CRC?

Checksum can miss errors, especially if the errors result in the same sum. CRC uses polynomial mathematics, which is more sensitive to a wider range of errors.

Where are Checksum and CRC used?

Checksum is commonly used in simpler error-checking systems, like IPv4 packet headers, while CRC is used in data communication protocols, such as Ethernet, and in data storage systems.